

Relazione finale

1. Obiettivo

L'obiettivo del mio progetto di tirocinio è stato la progettazione e lo sviluppo di una piattaforma software per il supporto alla gestione di casi clinici in ambito emergenziale, basata su un'architettura multi-agente AI orchestrata tramite n8n.

Il progetto è stato concepito come un sistema in grado di simulare il flusso operativo di un caso di emergenza sanitaria, dalla raccolta iniziale dei dati pre-ospedalieri fino alla formulazione di una sintesi clinica finale. A questo scopo sono stati definiti più agenti specializzati, ciascuno con un ruolo distinto nel processo di analisi e interpretazione del caso: CLARA per la raccolta e strutturazione dei dati iniziali, ARES per la stratificazione del rischio, VITA per la sintesi clinica, ATHENA per il ragionamento diagnostico avanzato ed EVAN come interfaccia clinica conversazionale finale.

Accanto alla componente di orchestrazione, è stato necessario sviluppare un backend applicativo in Flask e una dashboard web dedicata, così da consentire la visualizzazione dell'evoluzione del caso, la consultazione degli output generati dagli agenti e l'interazione con un avatar clinico virtuale.

Il fine ultimo del lavoro è stato quindi quello di realizzare un prototipo funzionante, capace di integrare AI generativa, workflow automation e interfacce web in un unico sistema dimostrativo orientato al dominio sanitario emergenziale.

2. Materiale

Il materiale utilizzato durante il tirocinio è stato costituito principalmente dalla documentazione tecnica delle tecnologie impiegate e dagli strumenti di sviluppo adottati per l'implementazione del prototipo.

In particolare, le tecnologie principali utilizzate sono state:

- Python, per lo sviluppo del backend applicativo;
- Flask, per la realizzazione delle API REST e per il rendering delle interfacce web;
- n8n, per l'orchestrazione dei workflow multi-agente;
- OpenAI API, per la generazione dei contenuti testuali e vocali associati agli agenti;
- HTML, CSS e JavaScript, per la costruzione della dashboard e delle interfacce utente;
- model-viewer, per l'integrazione del modello 3D dell'avatar;
- Rhubarb Lip Sync, per la sincronizzazione labiale dell'avatar con l'audio generato.

Dal punto di vista progettuale, è stata inoltre fondamentale la fase di studio relativa al prompt engineering, alla scomposizione di un problema complesso in agenti specialistici e all'integrazione di servizi eterogenei all'interno di un'architettura web distribuita.

3. Attività svolte

L'attività di tirocinio si è articolata principalmente nelle seguenti fasi:

- analisi del dominio applicativo e definizione del flusso clinico da simulare;
- progettazione dell'architettura software e del modello dati del caso clinico;
- sviluppo del backend Flask e delle API REST;
- realizzazione dei workflow multi-agente su n8n;
- sviluppo della dashboard web per la visualizzazione e l'interazione con il caso;
- integrazione dell'avatar clinico EVAN con output testuale e vocale;
- ottimizzazione dell'automazione e validazione del funzionamento complessivo del sistema.

3.1 Architettura della piattaforma e del backend

La piattaforma è stata progettata attorno al concetto di “caso clinico” come entità centrale del sistema. Ogni caso contiene i dati iniziali del paziente, gli output progressivi generati dagli agenti, lo stato della simulazione, la timeline delle fasi attraversate e, quando previsto, i dati ospedalieri e la priorità assegnata dal medico. Questo modello è stato implementato nel backend Flask attraverso una serie di route API dedicate alla creazione, lettura, aggiornamento e avanzamento del caso.

Sono state sviluppate API specifiche per:

- la creazione e lettura dei casi;
- il recupero dell'input destinato ai singoli agenti;
- il salvataggio degli output di CLARA, ARES, VITA, ATHENA;
- l'inserimento dei dati ospedalieri;
- l'avvio automatico della simulazione;
- la gestione della priorità clinica;
- l'interazione con l'avatar conversazionale.

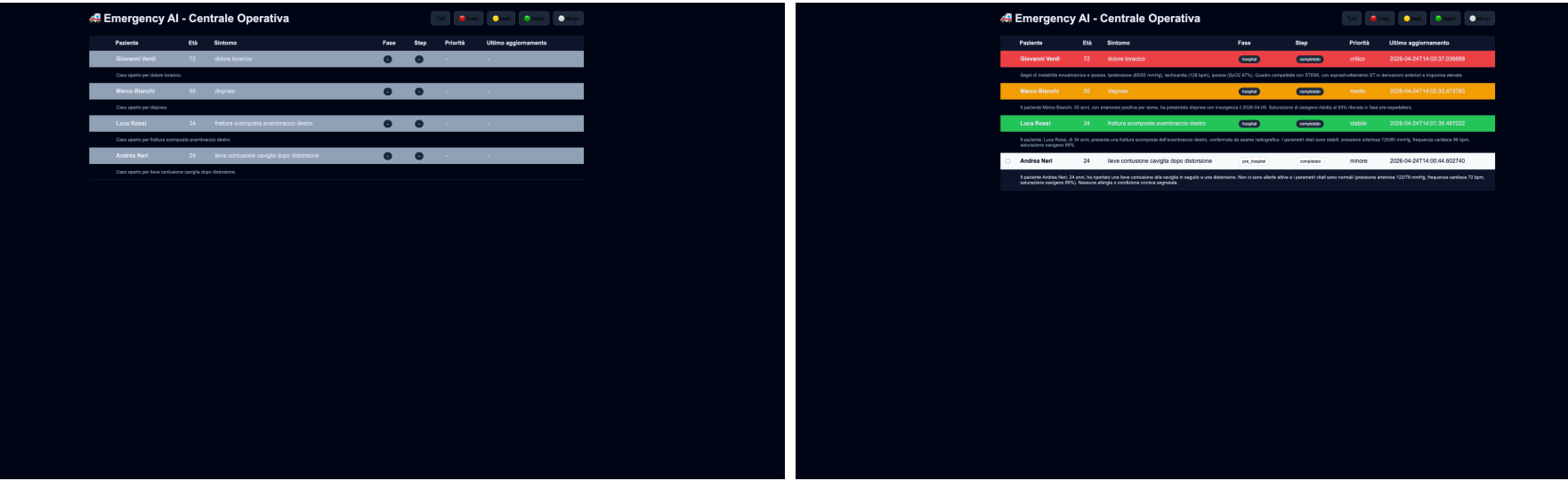
Una parte importante del lavoro ha riguardato l'automazione dei casi demo. Nel backend sono stati definiti diversi scenari preconfigurati, caratterizzati da livelli differenti di gravità e da percorsi distinti, così da simulare casi completi e casi limitati alla sola fase pre-ospedaliera. La logica di simulazione provvede ad avviare automaticamente i workflow, attendere i risultati, aggiornare lo stato del caso e determinare il completamento del processo.

Dal punto di vista architetturale, Flask è stato scelto per la sua leggerezza e per la facilità con cui consente di implementare API REST, integrarsi con servizi esterni e fungere sia da livello logico sia da punto di accesso per la dashboard web. Questa scelta si è dimostrata particolarmente adatta per un prototipo di tirocinio che richiedeva rapidità di sviluppo, flessibilità e semplicità di integrazione con n8n.

3.2 Dashboard clinica e interfaccia utente

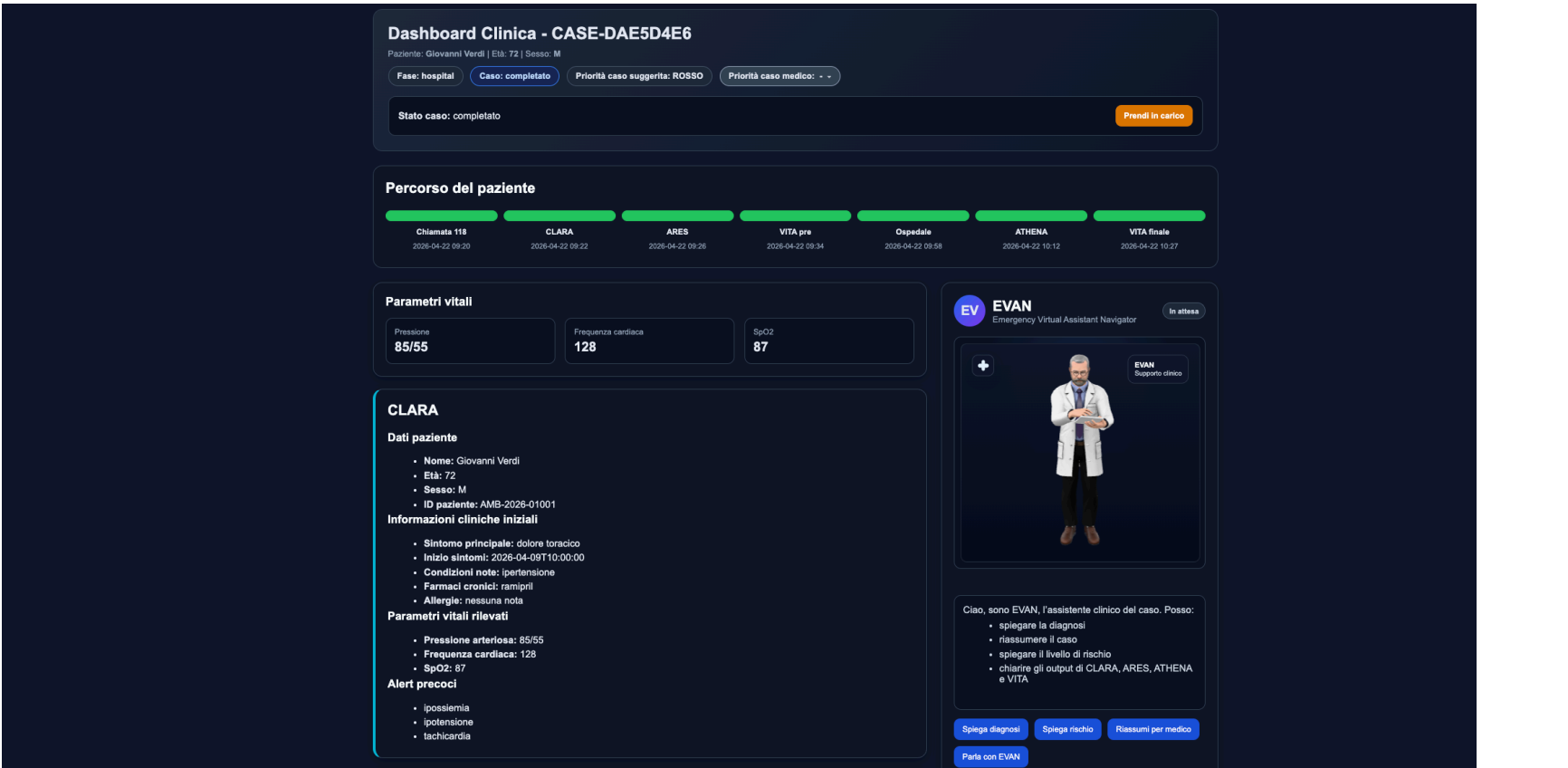
Parallelamente al backend è stata sviluppata una dashboard web con l’obiettivo di fornire una visualizzazione chiara e progressiva dell’evoluzione del caso clinico.

La home page della piattaforma è concepita come una “Centrale Operativa”, nella quale sono elencati i casi disponibili con informazioni sintetiche quali nome del paziente, età, sintomo principale, fase del percorso, step corrente e livello di priorità. Sono inoltre disponibili filtri rapidi per criticità, così da distinguere i casi critici, medi, stabili e minori. È presente anche un meccanismo di refresh automatico quando almeno un caso è in stato di elaborazione.



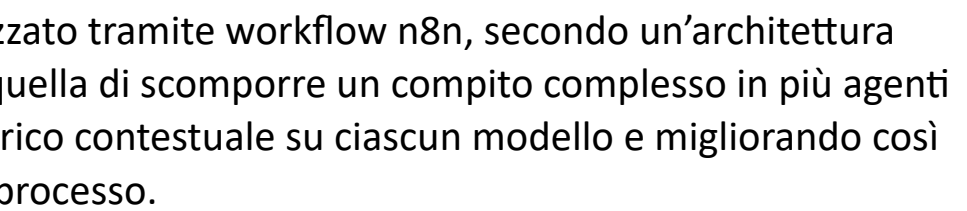
La dashboard del singolo caso mostra invece una vista dettagliata, suddivisa in più sezioni:

- intestazione con informazioni anagrafiche e stato della simulazione;
- timeline del percorso clinico;
- pannelli separati per gli output di CLARA, ARES, VITA e ATHENA;
- sezione dedicata ai dati ospedalieri;
- controllo della priorità suggerita dal sistema e della priorità eventualmente impostata dal medico;
- area avatar per l’interazione conversazionale.



È stata inoltre implementata la possibilità di avviare o riavviare la presa in carico del caso e di impostare manualmente una priorità clinica attraverso un controllo dedicato, permettendo così di simulare anche l'intervento decisionale del medico sul caso.

Il nucleo logico della piattaforma è stato realizzato tramite workflow n8n, secondo un'architettura multi-agente specializzata. L'idea di fondo, è quella di scomporre un compito complesso in più agenti con responsabilità circoscritte, riducendo il carico contestuale su ciascun modello e migliorando così sia la qualità delle risposte sia il controllo del processo.



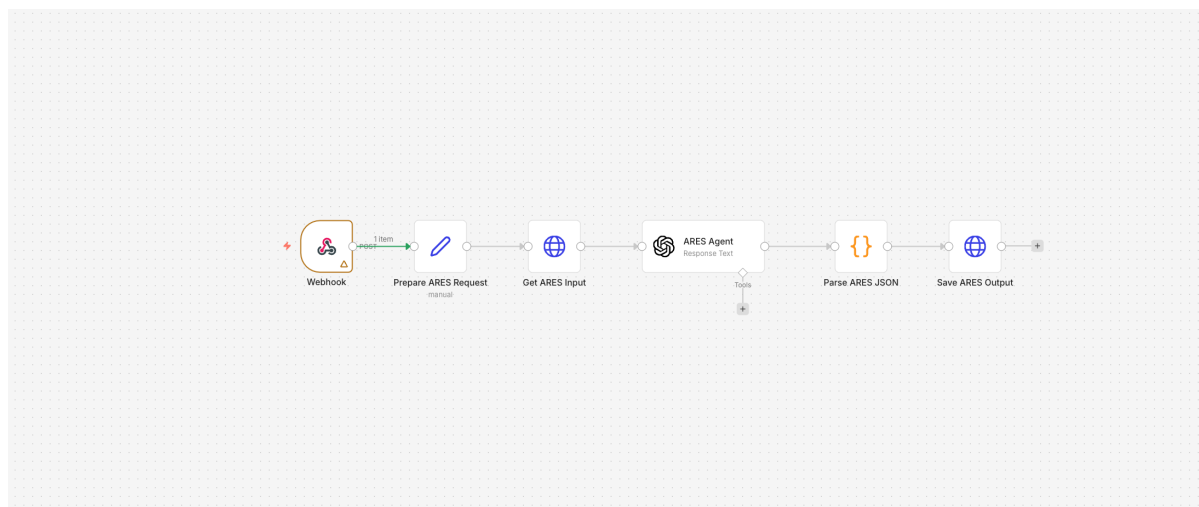
CLARA rappresenta il primo livello di elaborazione del caso. Il suo compito è acquisire i dati iniziali del paziente, strutturarli in un formato coerente e produrre un primo quadro clinico comprensivo di identificazione del paziente, sintomo principale, anamnesi essenziale, parametri vitali ed eventuali alert precoci. Nel backend, l'output di CLARA viene salvato come primo step strutturato del caso e determina il passaggio verso ARES.

3.3.2 Agente 2: ARES (Assessment, Risk Evaluation and Stratification Agent)



ARES è l'agente responsabile del pre-triage e della stratificazione del rischio. A partire dall'output di CLARA, produce un livello di rischio immediato, una priorità suggerita e una lista di scenari probabili con relative azioni raccomandate. Il sistema salva tale output e aggiorna il caso allo step successivo, cioè l'elaborazione VITA pre.

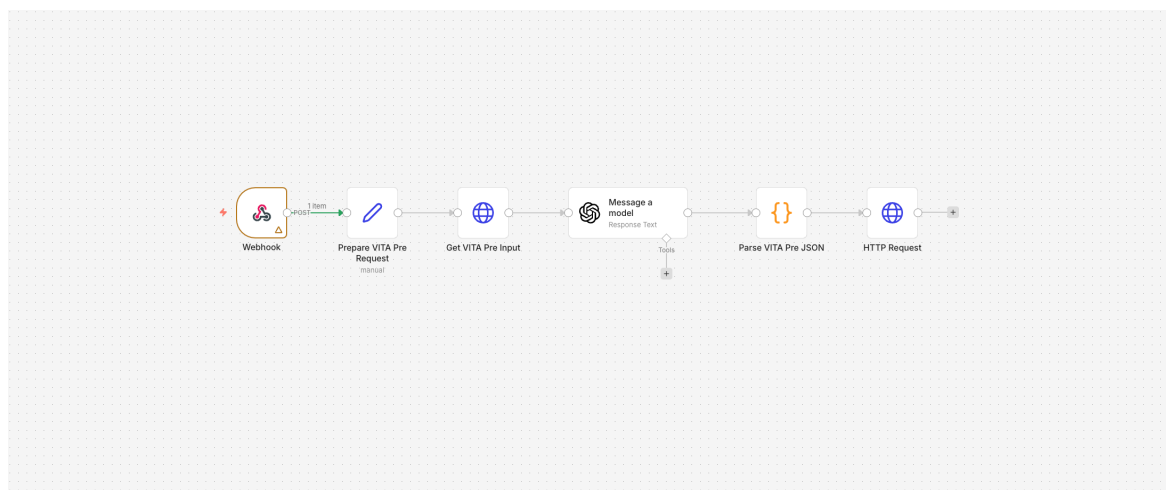
ARES svolge quindi una funzione cruciale di classificazione iniziale del caso, utile sia per l'interpretazione clinica sia per la rappresentazione sintetica nella dashboard.



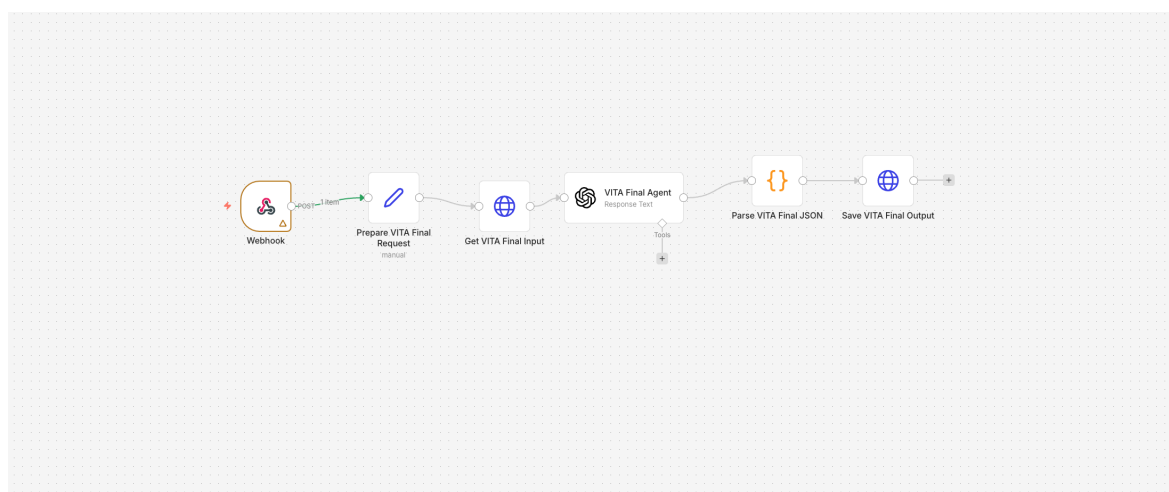
3.3.3 Agente 3: VITA (*Virtual Interpretation and Triage Assistant*)

VITA opera in due momenti distinti del flusso.

Nella prima fase, denominata VITA pre, sintetizza i dati pre-ospedalieri integrando quanto prodotto da CLARA e ARES. In questa fase genera un dashboard summary, una spiegazione operativa e un commento clinico. Se il caso è limitato alla fase territoriale, il workflow può terminare qui; in caso contrario, il sistema si predispone ad accogliere i dati ospedalieri.



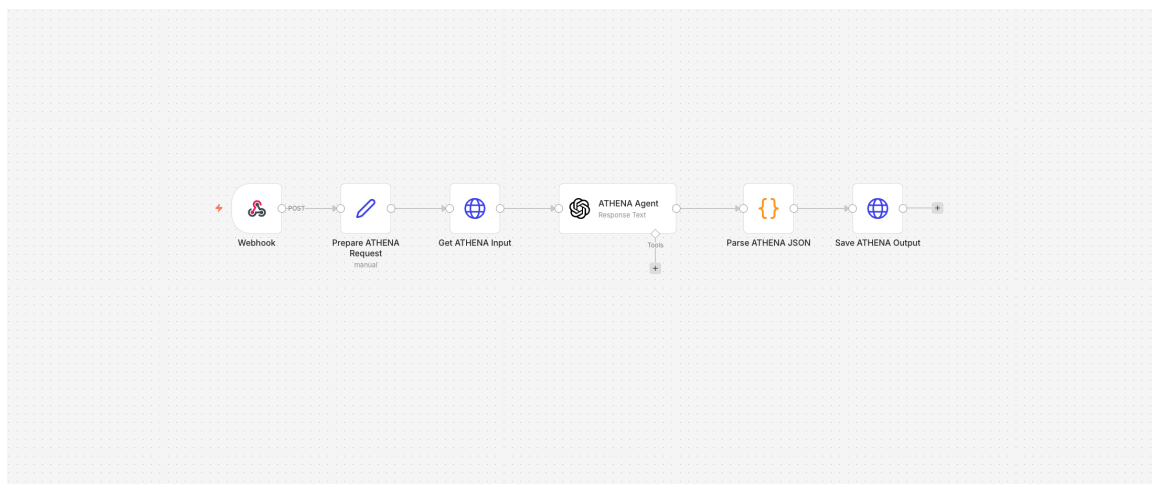
Nella seconda fase, VITA finale integra anche l'output di ATHENA e i dati ospedalieri, producendo una sintesi clinica conclusiva che rappresenta il punto di arrivo dell'intera simulazione.



3.3.4 Agente 4: *ATHENA (Advanced Therapeutic and Heuristic Evaluation for Next-step Analysis)*

ATHENA interviene dopo l'inserimento dei dati ospedalieri e svolge una funzione di approfondimento diagnostico. Produce un'ipotesi principale, una diagnosi differenziale e un insieme di evidenze a supporto. Tali informazioni vengono poi mostrate nella dashboard e passate a VITA finale per la composizione della sintesi conclusiva.

ATHENA rappresenta quindi il livello di ragionamento più avanzato dell'architettura, arricchendo il flusso con una componente interpretativa più vicina al ragionamento clinico specialistico.



3.4 Avatar clinico conversazionale: *EVAN (Emergency Virtual Assistant Navigator)*

Una delle estensioni più significative del progetto è stata la realizzazione dell'avatar clinico EVAN, concepito come interfaccia finale di spiegazione e consultazione del caso.

EVAN è integrato nella dashboard del singolo paziente e riceve come contesto l'intero stato del caso, inclusi gli output di tutti gli agenti precedenti. Attraverso l'endpoint dedicato, il backend costruisce un prompt di sistema che definisce EVAN come assistente clinico avanzato, incaricato di integrare i dati prodotti da CLARA, ARES, VITA e ATHENA e di rispondere in modo rigoroso, chiaro e coerente con il caso corrente.

Dal punto di vista dell'interfaccia, EVAN può:

- rispondere a domande scritte;
- ricevere input vocale;
- generare audio sintetico della risposta;
- sincronizzare la riproduzione audio con un'animazione facciale e con il tracciamento delle frasi mostrate a schermo.

L'aggiunta di EVAN ha arricchito il progetto su due livelli. Da un lato ha migliorato l'usabilità e la qualità dimostrativa della piattaforma; dall'altro ha introdotto un'interfaccia più vicina a un possibile scenario reale di supporto al medico, in cui il sistema non si limita a mostrare dati ma li interpreta e li comunica in forma naturale.

4. Conclusioni

A conclusione del lavoro svolto, è stato realizzato un prototipo software multi-agente pienamente funzionante, capace di simulare il percorso di gestione di un caso clinico di emergenza attraverso una pipeline composta da raccolta dati, stratificazione del rischio, sintesi clinica intermedia, approfondimento diagnostico e sintesi finale. Il sistema integra in modo coerente backend Flask, workflow n8n, dashboard web e interfaccia conversazionale avanzata.

L'architettura sviluppata si è dimostrata modulare, estendibile e adatta a rappresentare in modo efficace un flusso complesso. La scomposizione in agenti distinti ha permesso di separare le responsabilità logiche del sistema, migliorando la leggibilità del processo e il controllo sugli output. L'integrazione dell'avatar EVAN ha inoltre aggiunto una componente innovativa di interazione uomo-macchina, rendendo il prototipo più completo sia dal punto di vista tecnico sia da quello applicativo.

Dal punto di vista formativo, il progetto mi ha permesso di consolidare competenze di backend development, progettazione di API REST, sviluppo frontend, orchestrazione di workflow, prompt engineering e integrazione di modelli generativi in un'applicazione completa. Ho inoltre avuto modo di confrontarmi con problematiche concrete di sincronizzazione tra componenti, gestione dello stato, automazione dei processi e progettazione di interfacce orientate all'utente finale.

Tra i possibili sviluppi futuri del sistema si possono individuare:

- l'integrazione con basi dati cliniche o servizi sanitari reali;
- il supporto a un numero maggiore di casi simultanei;
- l'aggiunta di moduli di audit e tracciabilità delle decisioni;
- l'estensione della logica multi-agente a contesti specialistici differenti;
- il miglioramento dell'interfaccia avatar con componenti multimodali ancora più evolute.

Nel complesso, l'esperienza di tirocinio presso CoRiTel in Ericsson si è rivelata estremamente formativa, permettendomi di partecipare allo sviluppo di un progetto avanzato, multidisciplinare e vicino a problematiche reali di integrazione tra AI e sistemi software.

5. Modalità di svolgimento

Il tirocinio è stato svolto presso CoRiTel in Ericsson e si è sviluppato attraverso una combinazione di attività di analisi, progettazione, sviluppo e validazione del prototipo. I team hanno garantito supporto tecnico e formativo efficace, monitorando ogni progresso e con molti suggerimenti che hanno portato a grandi miglioramenti.

L'attività si è basata su una modalità di lavoro progressiva: una prima fase di studio e definizione dell'architettura, una seconda fase di implementazione del backend e dei workflow, una terza fase di

sviluppo della dashboard e dell'avatar, e infine una fase conclusiva di ottimizzazione e verifica del funzionamento integrato dell'intero sistema.

Durante il tirocinio ho potuto lavorare sia sugli aspetti architetturali sia su quelli implementativi, occupandomi direttamente dello sviluppo del backend Flask, della progettazione delle route API, dell'integrazione con n8n, della costruzione dell'interfaccia web e della definizione delle logiche di interazione tra agenti. Il lavoro ha richiesto continui test incrementali, correzione di errori di integrazione e perfezionamento dell'automazione del flusso.

L'esperienza ha rappresentato un'occasione concreta per applicare competenze teoriche a un progetto reale, affrontando problematiche tipiche dello sviluppo software moderno: modularità, interoperabilità, orchestrazione di servizi, integrazione con modelli AI e progettazione di interfacce orientate alla spiegazione e al supporto decisionale.